



SOFTWARE REQUIREMENT SPECIFICATION

Project Name: Ralsei Coach House System

Project Code: RCHS

Table of Contents

I. Record of Changes.....	3
II. Software Requirement Specification.....	4
1. Overall Requirements.....	4
1.1 Context Diagram.....	4
1.2 Main Business Processes.....	4
1.3 User Requirements.....	4
1.4 System Functionalities.....	12
1.5 Entity Relationship Diagram.....	15
2. Use Case Specifications.....	17
2.1 Booking ticket.....	17
2.1.4 Package booking.....	22
2.2 Xyz Feature.....	22
3. Functional Requirements.....	23
3.1 Feature Name1.....	23
3.2 User Authentication.....	23
3.3 System Administration.....	24
4. Non-Functional Requirements.....	25
4.1 External Interfaces.....	25
4.2 Other Requirement.....	25
5. Requirement Appendix.....	26
5.1 Business Rules.....	26
5.2 System Messages.....	27
5.3 Other Requirements.....	27

I. Record of Changes

Date	A* M, D	In charge	Change Description
25/5	A	Đức	1.3.1 Actors 1.3.2 Use Case Diagrams
25/5	A	Thành	1.1 Context Diagram 1.4.1 Screens Flow
28/5	M, A	Hà	1.3.1 Actors 1.3.2 Use Case Diagrams 1.5 Entity Relationship Diagram
02/06	M	Hà	1.5 Entity Relationship Diagram
08/07	M	Hà	1.3 User Requirements

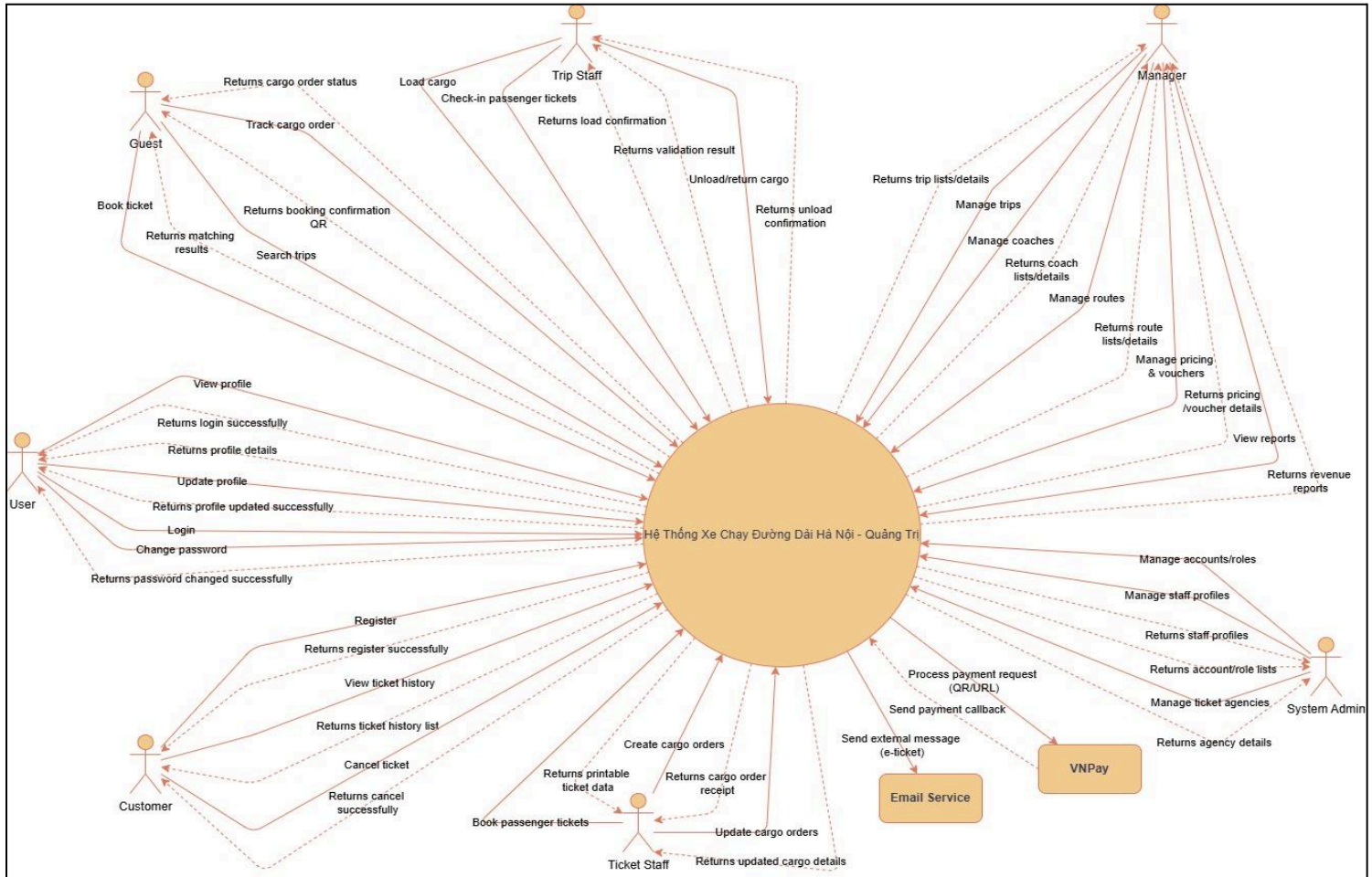
*A - Added M - Modified D - Deleted

II. Software Requirement Specification

1. Overall Requirements

1.1 Context Diagram

Thiếu mô tả giới thiệu về dự án/hệ thống nhóm đang làm này



1.2 Main Business Processes

[Provide main workflows (business processes) using the swim-lane diagram(s) like sample below]

1.2.1 Online Passenger Booking Flow

1.3 User Requirements

1.3.1 Actors

#	Actor	Description
1	Guest	Unauthenticated visitor on the Customer Site who can search, filter, and book trips, make payments.

2	User	An abstract base actor representing any authenticated individual. Handles core account features including login, logout, password recovery, and personal profile management.
3	Customer	Registered passenger (specializes Guest and User). Can register via Firebase Authentication, book tickets online, complete online payment, view booking history (passenger tickets and cargo orders), open ticket details, cancel eligible tickets, and display QR codes when qualified.
4	Ticket Staff	Ticketing-office staff (specializes User). Manages passenger tickets (search, view detail, book at counter, cancel, change seat, transfer trip, update passenger info), manages cargo orders across lifecycle stages (view, create, assign to trip, track pending/in-transit/arrived, confirm delivery, collect payment), and views operational trip information.
5	Trip Staff	On-board operational staff such as drivers and attendants (specializes User). Views assigned trips, marks their assigned trip as in-progress or completed, performs pre-departure duties (passenger check-in, cargo loading), and performs arrival duties (cargo unloading).
6	Manager	Operations supervisor (specializes User). Manages trip scheduling, coach type configuration and pricing, individual coaches and seat layouts, route network and stop points, cargo catalog and freight pricing, voucher campaigns, and finance operations including pending refund review and confirmation.
7	System Admin	Highest-level administrator (specializes User). Manages system accounts (view, lock/unlock, reset password, assign roles), view roles, staff onboarding and profile/status updates, customer records, and ticket agency records.
8	SePay (Payment Gateway)	External payment provider. Receives payment requests from the system, processes online transactions, and sends payment callbacks (IPN/webhook) with final transaction status.
9	Email Service	External notification channel. Sends automated outbound messages such as ticket confirmations, cancellations, and other system-triggered emails to users.

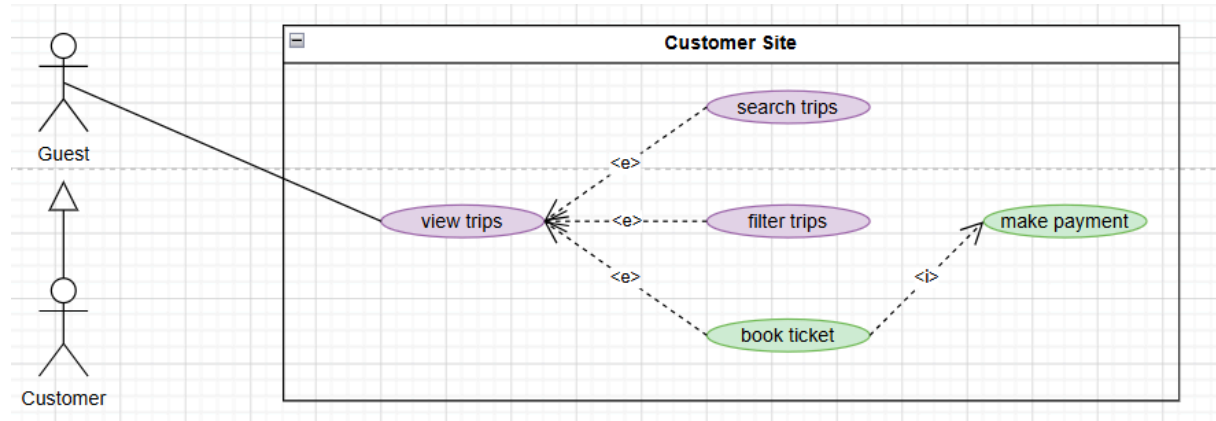
1.3.2 Use Cases (UC)

ID	Use Case	Feature	Use Case Description
01	View Menu	Feature Name1	

02	Order a Meal	Feature Name2	
03	...		

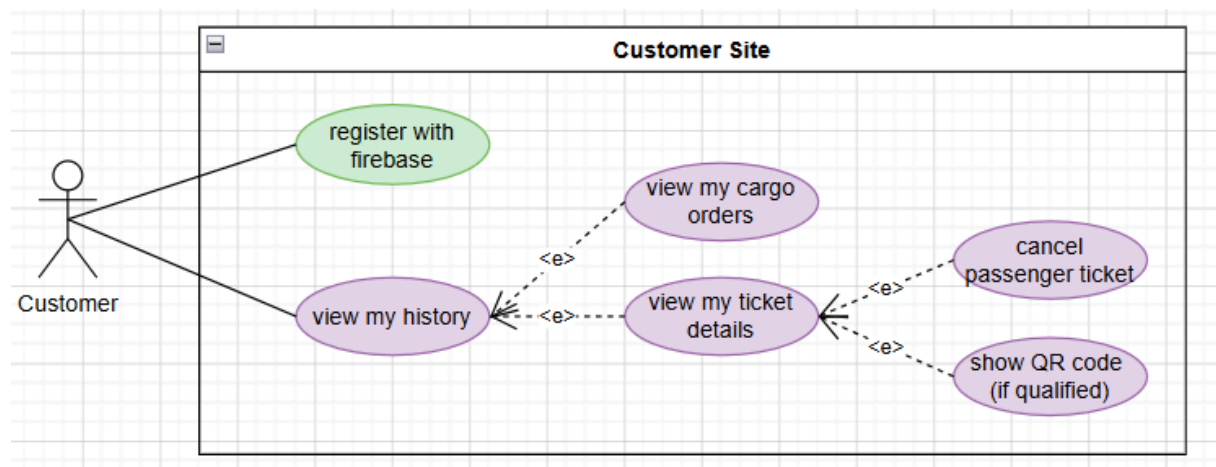
1.3.2 Use Case Diagrams

1.3.2.1 UCs for Guest

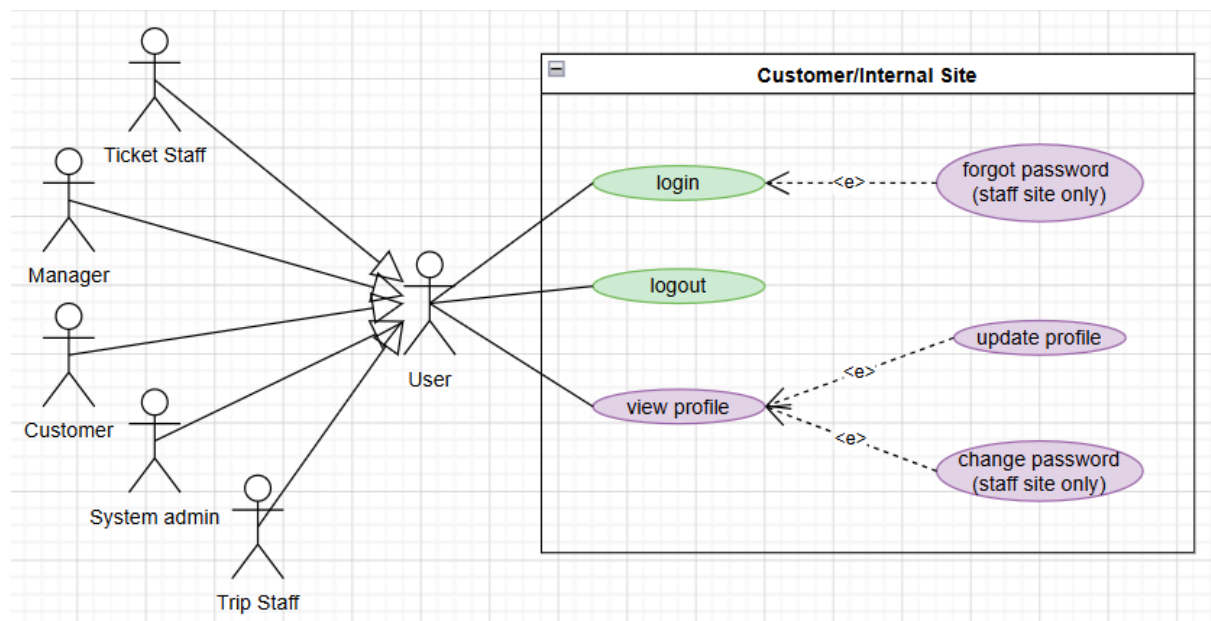


1.3.2.2 UCs for Customer

- Customer also extends all use cases of Guest.
- Customer also extends all use cases of User.

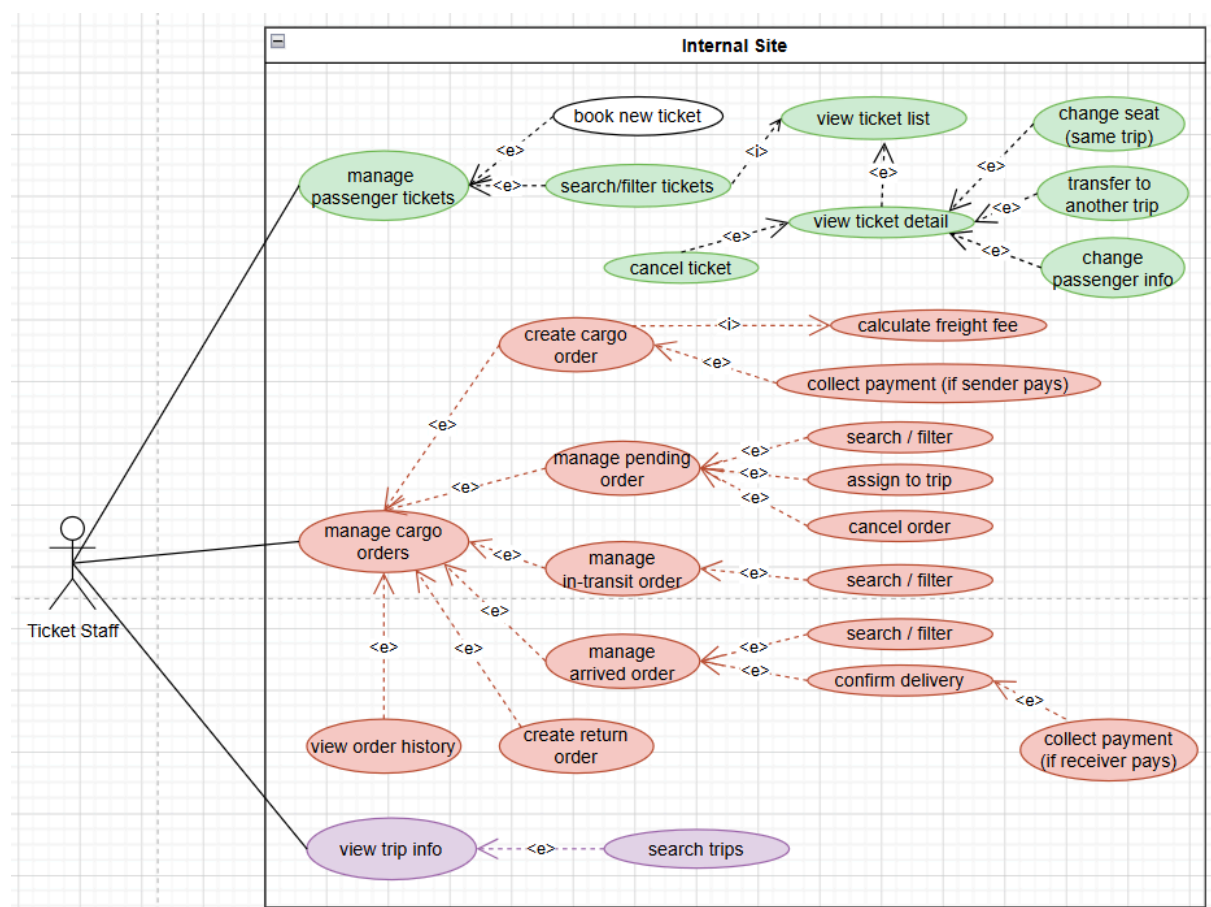


1.3.2.3 UCs for User



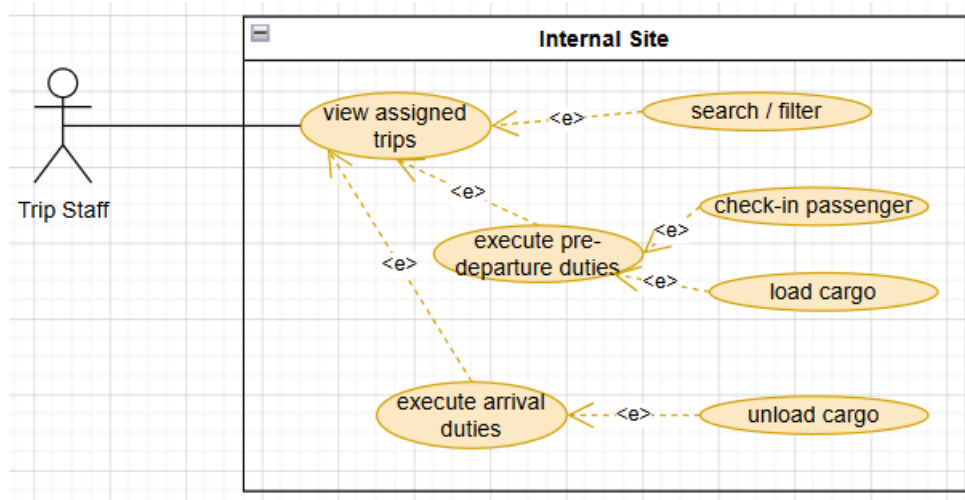
1.3.2.4 UCs for Ticket Staff

- Ticket Staff also extends all use cases of User.



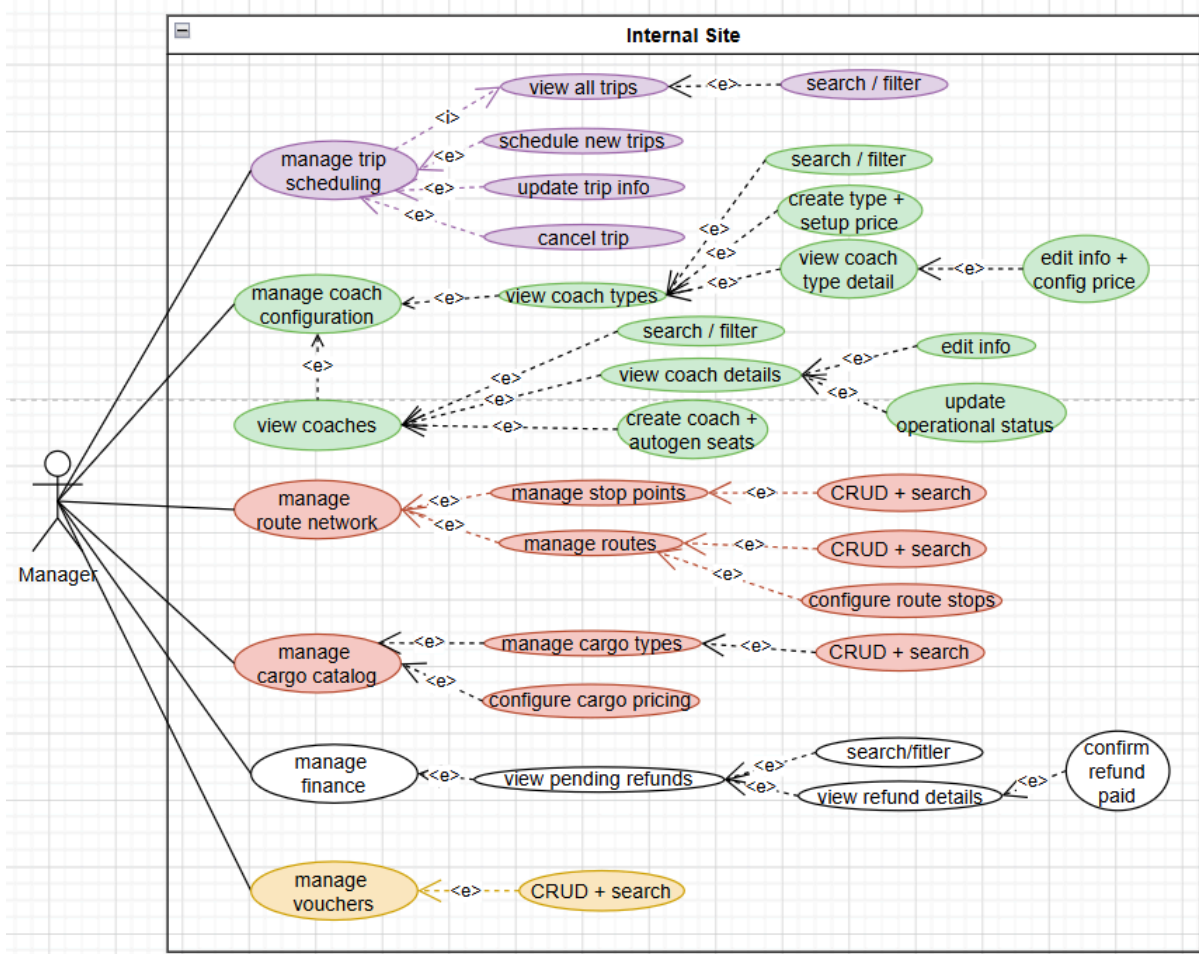
1.3.2.5 UCs for Trip Staff

- Trip Staff also extends all use cases of User.

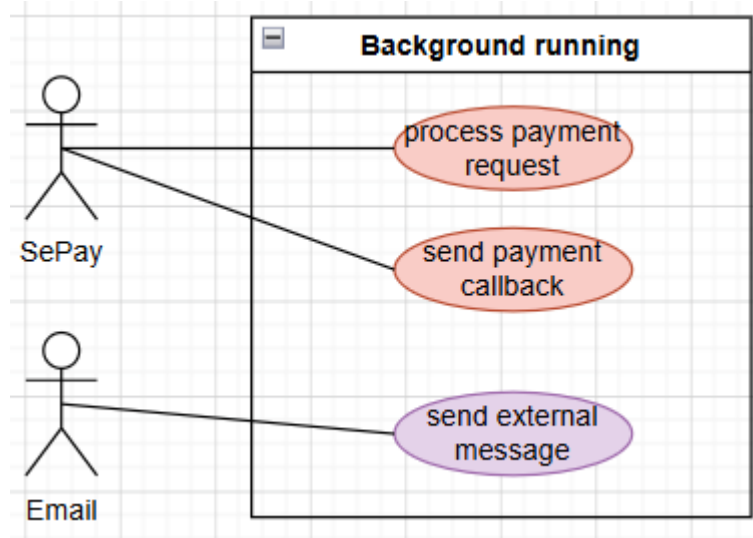


1.3.2.6 UCs for Manager

- Manager also extends all use cases of User.

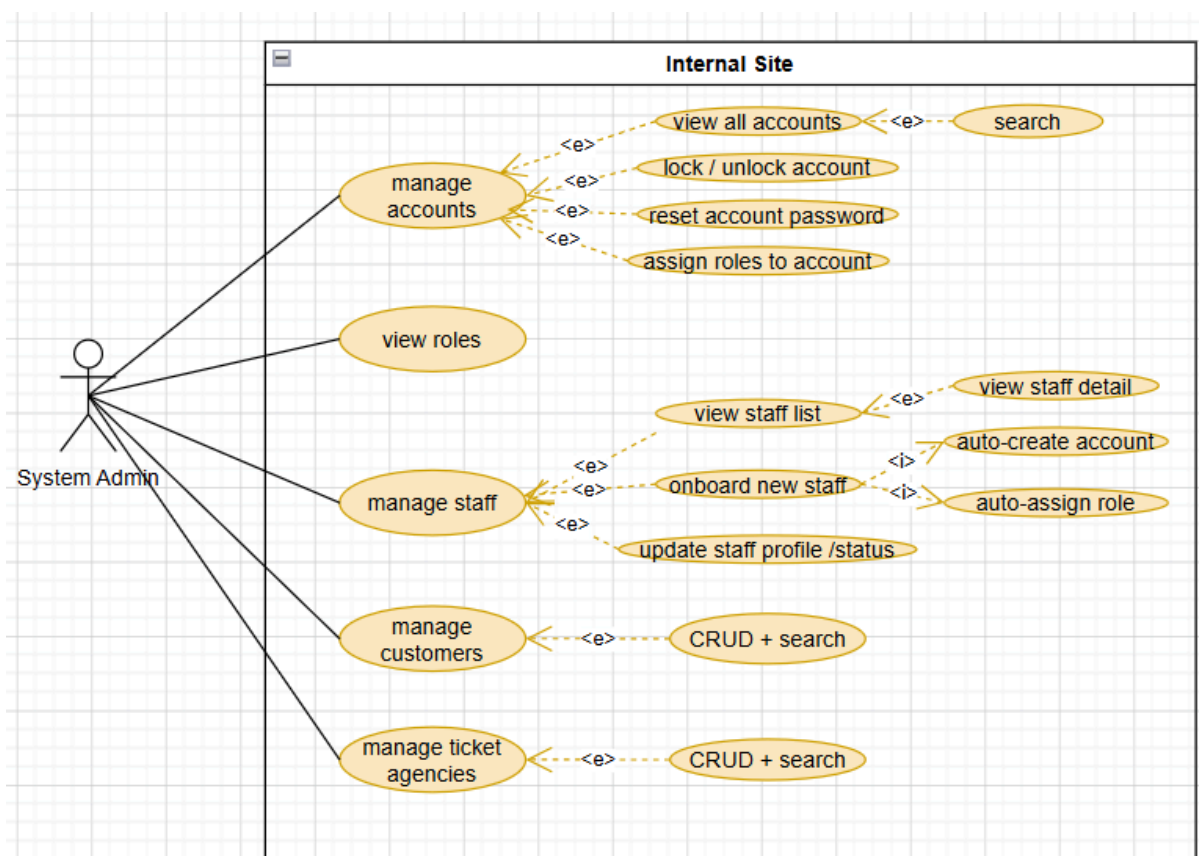


1.3.2.7 UCs for External Actors



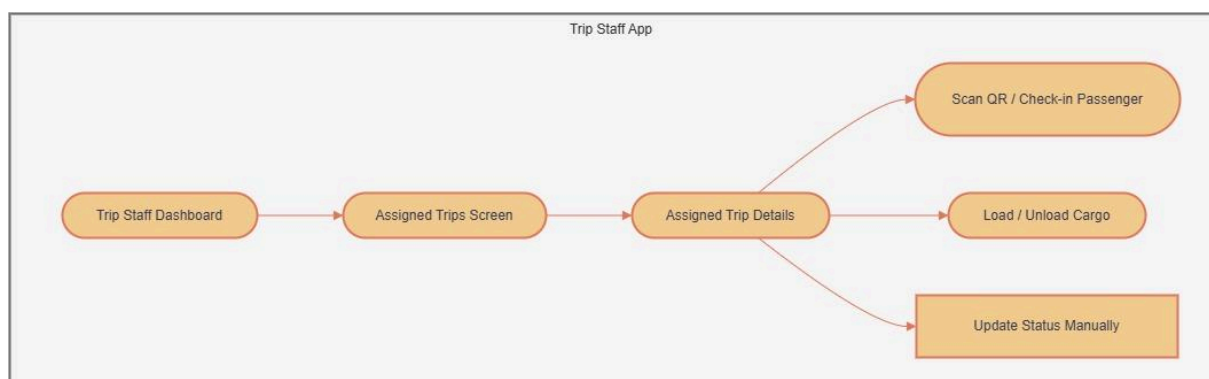
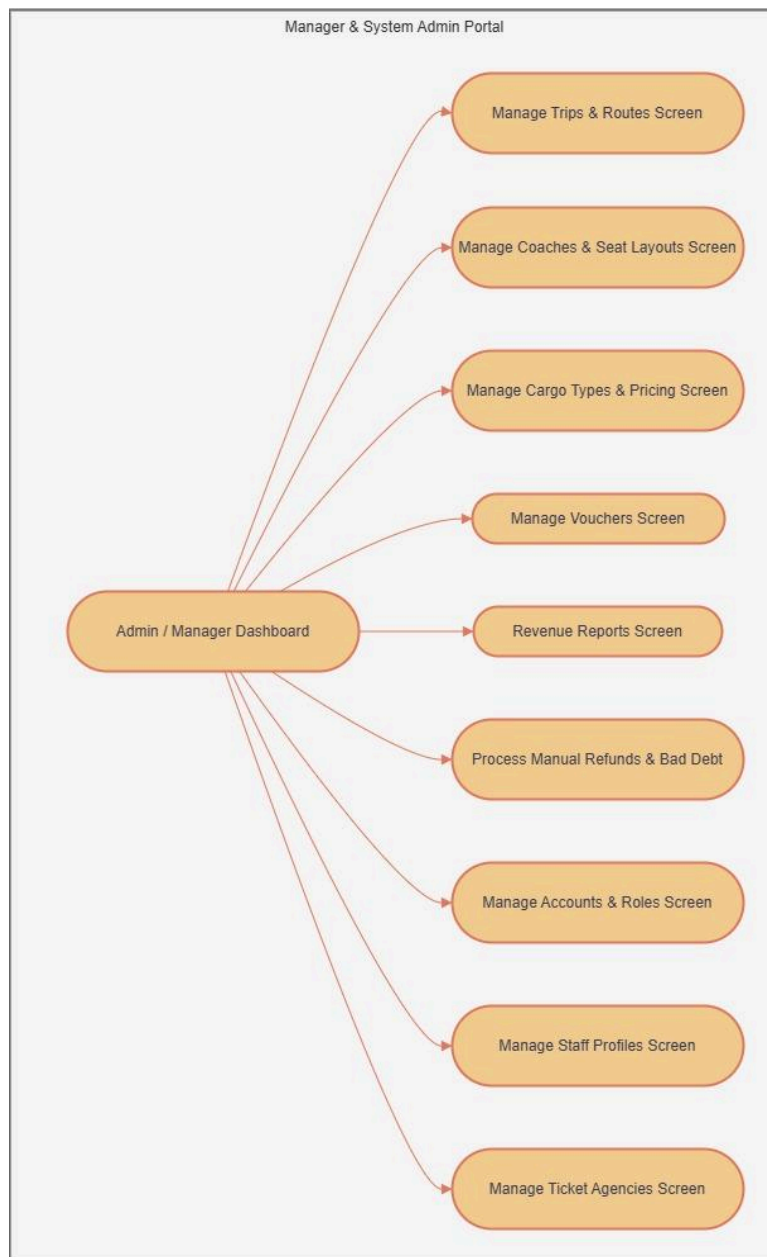
1.3.2.8 UCs for System Admin

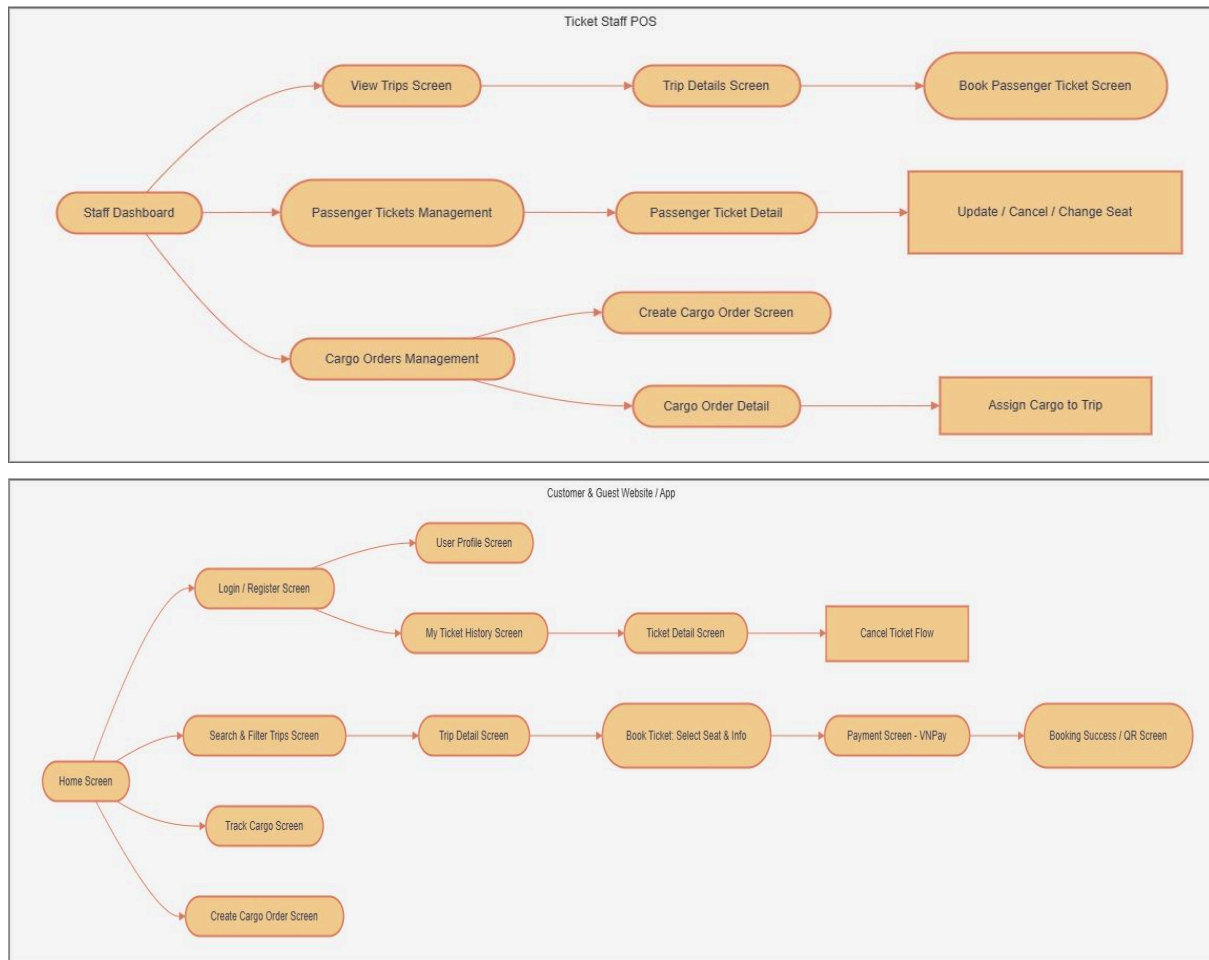
- System Admin also extends all use cases of User.



1.4 System Functionalities

1.4.1 Screens Flow





1.4.2 Screen Authorization

[Provide the system roles authorization to the system features (down to screens, and event to the screen activities if applicable) in the table form as below – replace Role-Name1, Role-Name2, ... with your specific system user role names]

Screen	Role-Name 1	Role-Name 2	Role-Name3	...
<<Screen Name1>>	X		X	X
<<Screen Name2>>		X		
...				

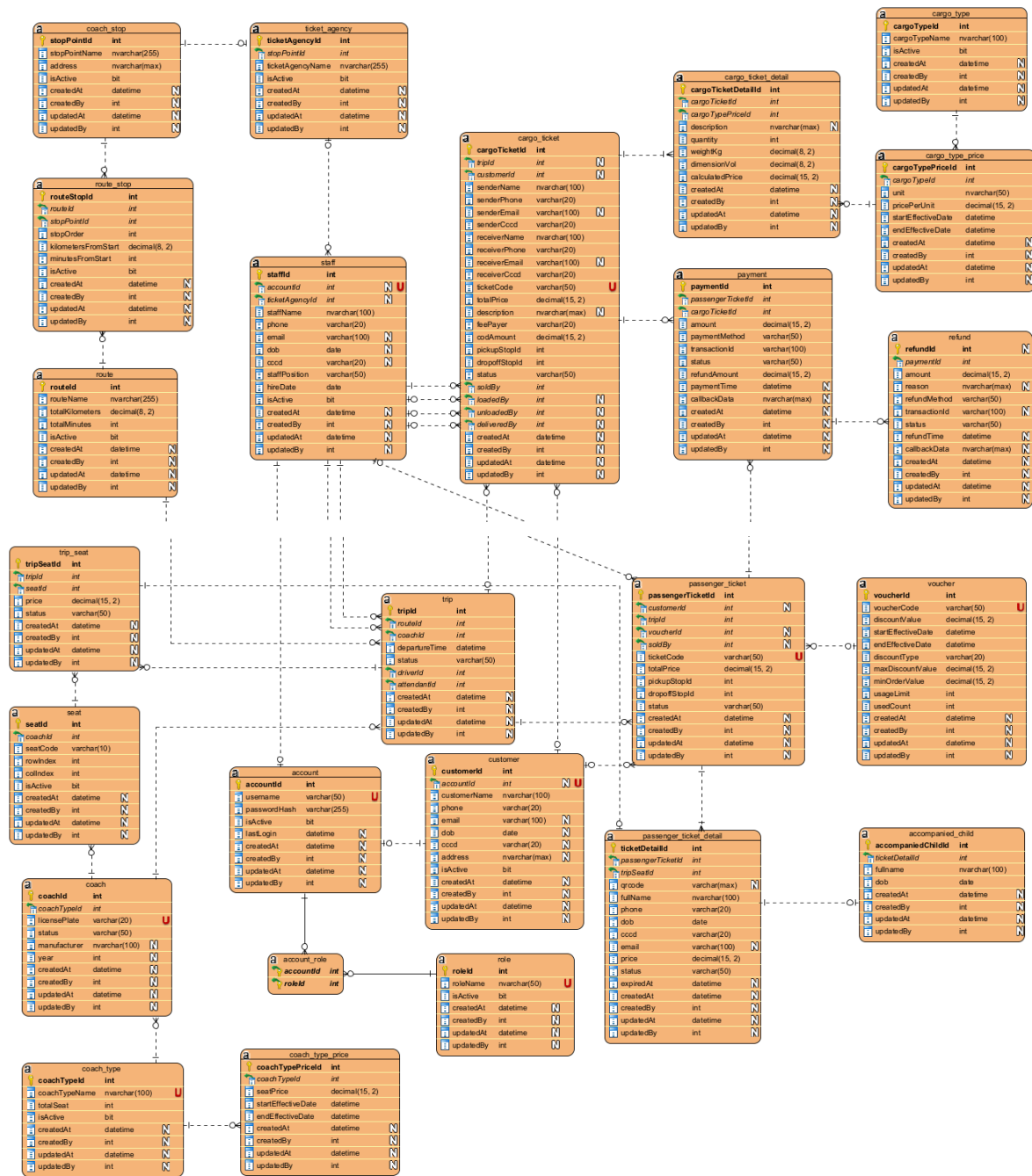
1.4.3 Non-UI Functions

[Provide the descriptions for the non-screen system functions, i.e batch/cron job, service, API, etc.]

#	Feature	System Function	Description
1	<<Feature Name>>	<<Function Name1>>	<<Function Name1 Description>>

2	...		
---	-----	--	--

1.5 Entity Relationship Diagram



Entities Description

#	Entity	Description
1	Account	Stores authentication credentials (username/phone, password hash, status) and handles login lifecycles. Strictly maps to either one Customer or one Staff member.

2	Role	Defines the access control levels within the system (e.g., SYSTEM_ADMIN, MANAGER, TICKET_STAFF, CUSTOMER) for authorization purposes.
3	Customer	Contains the personal profile and identification details (name, phone, email, CCCD) of external users and passengers.
4	Staff	Stores employee records, including their job position, hire date, and the specific TicketAgency they are assigned to (if applicable).
5	TicketAgency	Represents the physical ticket offices or branches operated by the company, linked to a specific physical CoachStop.
6	CoachStop	Defines geographical, physical stopping points (addresses, stations, toll gates) where buses can pick up or drop off passengers and cargo.
7	RouteStop	An associative entity that maps a CoachStop to a Route, defining the exact stop order, distance from the start (km), and estimated travel time.
8	Route	Represents a logical path or journey connecting multiple predefined stops from a starting terminal to a destination.
9	CoachType	Categorizes vehicle models (e.g., 20-cabin VIP Limousine, 32-bed Luxury Sleeper) and defines the total seating capacity for that specific type of vehicle.
10	CoachTypePrice	Manages the base ticket pricing for a specific CoachType, including effective date ranges to support dynamic pricing for holidays or seasons.
11	Seat	Represents an individual, physical seat or bed permanently attached to a specific Coach, containing exact coordinates (row, column) and seat codes (e.g., A1, B2).
12	Coach	Represents a physical vehicle in the fleet, containing its license plate, manufacturer, operational status, and acts as the parent entity for its specific physical seats.
13	AccompaniedChild	Stores details of infants or small children traveling free of charge, linked to a primary PassengerTicketDetail.
14	PassengerTicketDetail	Represents an individual seat reservation, holding the specific occupant's details (name, CCCD) and maintaining its own independent lifecycle status. It links directly to a TripSeat to ensure historical price and location integrity.
15	PassengerTicket	The master transaction record for a passenger booking. It aggregates multiple ticket details, total calculated price, and applied vouchers.

16	CargoTypePrice	Defines the specific pricing rules (per kg, per m3, or per item) for different cargo types across defined timeframes.
17	CargoType	Categorizes the types of transportable goods the company accepts (e.g., standard packages, motorcycles, fragile items).
18	CargoTicketDetail	Represents an individual package (grouping items as a package of same cargo type), storing its dimensions, weight, and individually calculated freight price.
19	CargoTicket	The master operational record for a cargo shipment, storing sender/receiver info, payment responsibilities, and tracking its logistics lifecycle (received to delivered).
20	Payment	Tracks the financial transactions (VNPay, cash, bank transfer) associated with a master PassengerTicket or CargoTicket, recording the status and transaction IDs.
21	Refund	Manages money returned to customers due to partial/full ticket cancellations or manual service adjustments, referencing the original Payment.
22	Trip	An operational instance of a Coach driving a specific Route at a scheduled departure time, managed by an assigned driver and attendant.
23	Voucher	Defines promotional discount codes, their application rules (percentage vs. fixed amount, usage limits, minimum order values), and validity periods.
24	TripSeat	An operational snapshot entity that tracks the real-time availability (available, locked, sold) and exact price of a physical Seat for a specific scheduled Trip.

2. Use Case Specifications

Chi specify complex UCs involving in the main workflows (business processes)

2.1 Xyz Feature

2.1.1 Book ticket feature

Use case ID, Name	UC01 - Book ticket		
Primary Actors	Customer	Secondary Actors	Sepay, Email
Description	As a traveler, I want to book a coach ticket, select the date, time, location I want to start/stop to manage my trip easily. I also want to book the seat, type of the coach and pay through QR code or bank transfer so that i can secure my seat without calling hotline or worrying about double booking		
Preconditions	1. The user has accessed the Ralsei Coach House booking website 2. Active bus routes, schedules and seat inventories are synchronized and available in the system.		
Postconditions	1. The selected seat status is updated to "Booked" in the global database. 2. Financial transactions are logged, and revenue is updated on the Owner's dashboard. 3. A unique e-ticket with a QR code is generated and sent to the customer.		
Normal Sequence/Flow	1. The customer search trip using the search bar at the home page to select date and start/end point.(See A6.1 and A6.2) 2. The system will display all trips in the day that user select (See A5.1) 3. Customer request booking ticket at view trips 4. Customer select seats 5. The system show book ticket form (See A2.1 and 2.2) 6. Customer confirm the booking 7. The system calculate money total 8. The system generate QR code for booking 9. The customer scan QR code to make payment (See A4.1 and A4.2) 10. The system show confirm “booking is successful”, save booking to database, send message to confirm		

	11. An Email with ticket information will be send to customer email, use case end with no error
Alternative Sequences/Flows	Alternative Flow A2.1 get Customer information (if customer is logged in) 1. The system get Customer information in the account and auto fill ticket form using account data , continuing to step 5. A2.2. Customer is not login 1. The customer enter customer informations 2. The system verify send OTP Message customer info (Phone Number) 3. The customer enters OPT. 4. Does the system check if the OTP is correct?(See A3.1 and A3.2). 5. continuing to step 5. A3.1. Customer enter the correct OTP 1. The system will approve the phone number. 2. The customer will continue to fill information. 3. Return to step 4 (A2.2). A3.2 Customer enter the incorrect OTP 1. The system will reject the phone number. 2. The customer must correct the correct phone number so the system can continue. 3. return to step 4. A4.1 The customer complete the QR Payment 1. The system will accept the payment and add booking for customer 2. Continue to step 9. A4.2 The customer failed the QR Payment 1. The system will reject the payment and reject the booking request. 2. The customer will get a “Failed” notification and will be sent back to the home screen. Use case end with no error. A5.1 The customer use filter trip

	1. The system will display according to price, coach type, start/end time, return to step 2 A6.1 The customer using the past search and the date still has not come yet. 1. The system will display the result like normal. Continue to step
--	---

===== NHẬP

2.1 *Booking ticket*

2.1.1 Booking ticket

Use case ID, Name	UC01 - Book ticket		
Primary Actors	Customer	Secondary Actors	Sepay
Description	As a traveler, I want to book a coach ticket, select the date, time, location I want to start/stop to manage my trip easily. I also want to book the seat, type of the coach and pay through QR code or bank transfer so that i can secure my seat without calling hotline or worrying about double booking		
Preconditions	1. The user has accessed the Ralsei Coach House booking website 2. Active bus routes, schedules and seat inventories are synchronized and available in the system.		
Postconditions	1. The selected seat status is updated to "Booked" in the global database. 2. Financial transaction is logged, and revenue is updated on the Owner's dashboard. 3. A unique e-ticket with a QR code is generated and sent to the customer.		

Normal Sequence/Flow	1. Customer request booking ticket at view trips 2. The system show book ticket form (See A2.1) 3. Customer select seats 4. The system calculate money total 5. Customer Select booking (See A5.1) 6. The system generate QR code for booking 7. Customer scan QR code to make payment 8. The system show confirm “booking is successfull” , save booking to database, send message to confirm
Alternative Sequences/Flows	Alternative Flow A2.1 get Customer information (if customer is logged in) 1. The system get Customer information and show ticket form, return step 3. A5.1. Customer is not login 6. The system show fields to customer enter customer information. 7. The customer enter customer informations (?) 8. The system verify send OTP Message customer info (Phone Number) 9. Customer enter OPT 10. The system check (See ?...) 11. Return step 6

Primary and Secondary Actors

An actor is a person or other entity external to the software system being specified who interacts with the system and performs use cases to accomplish tasks. Name the primary actor

that will be initiating this UC and any other secondary actors who will participate in completing execution of the UC.

Description

Provide a brief description of the reason for and outcome of this use case, or a high-level description of the sequence of actions and the outcome of executing the use case. The description can be in the form of a user story (As a <type of user>, I want <some goal> so that <some reasons>)

Preconditions

List any activities that must take place, or any conditions that must be true, before the use case can be started.

Postconditions

Describe the state of the system at the successful conclusion of the use case execution.

Normal Flow

Provide a description of the user actions and corresponding system responses that will take place during execution of the use case under normal, expected conditions.

Alternative Flows

Describe below two information if any:

- Other successful usage scenarios that can take place within this use case. State the alternative flow, and describe any differences in the sequence of steps that take place.
- Any anticipated error conditions that could occur during execution of the use case and how the system is to respond to those conditions.

2.2 Register/Login Feature

2.2.1 Login System

FIrebase auth

Use case ID, Name	UC01 - User Login		
Primary Actors	User, Guest	Secondary Actors	System, Database
Description	The user provides their phone number (username) and password on the login form. The system authenticates the credentials, determines the user's role, and redirects them to the appropriate landing page. If the user checks "Remember Me," their session persists across browser restarts.		
User Story	As a registered user, I want to log in with my phone number and password so that I can access role-specific features and manage my bookings.		

Preconditions	1. The user must have a registered account in the `account` table. 2. The account must have at least one role assigned in `account_role`. 3. The corresponding `role` must exist in the `role` table. 4. The backend server must be running on port 9090. 5. The database connection must be active.
Postconditions	On success: 1. The user's `lastLogin` field in the `account` table is updated to the current timestamp. 2. A random UUID token is generated and returned in the response. 3. The user's identity (username + role) is stored in `localStorage` or `sessionStorage` depending on "Remember Me." 4. The user is redirected to `/home`. On failure: 1. No session data is persisted. 2. An appropriate error message is displayed on the login form. 3. The user remains on the login page.
Normal Sequence/Flow	Main Flow 1. The user navigates to the Login page. 2. The system displays the login form. 3. The user enters their phone number and password. 4. The user optionally selects "Remember Me". 5. The user clicks the Sign In button. 6. The system validates the input fields. 7. The system sends the login request to the backend. 8. The system verifies the user credentials. 9. The system checks whether the account is active. 10. The system creates a login session and authentication token. 11. The system stores the user session. 12. The system updates the interface to reflect the logged-in state. 13. The system redirects the user to the homepage.
Alternative Sequences/Flows	A1 - Invalid Credentials (Step 8) 1. The system cannot verify the username or password. 2. The system displays an error message. 3. The user remains on the login page. A2 - Account Disabled (Step 9) 1. The system detects that the account is disabled. 2. The system displays an account locked message. 3. The user remains on the login page. A3 - Remember Me Checked 1. During the last login session, the user selected the "Remember Me" option. 2. After successful authentication, the system stores the user session in persistent browser storage. 3. The user remains logged in and the system redirect the user to homepage.
Exceptional Flow	E1 – Network or Server Failure 1. The login request cannot be completed because of a network or server issue. 1. The system displays a generic login failure message. 2. The user remains on the login page.

2.2.2 Register System

Use case ID, Name	UC-02: User Registration		
Primary Actors	User, Guest	Secondary Actors	System, Database
Description	A guest user registers a new account using a phone number and password in order to access the system and book bus tickets. The user provides required registration information and optional personal details. The system validates the input, checks whether the phone number already exists, creates the account, assigns the Customer role, and stores the customer profile.		
User Story	As a guest user, I want to register an account using my phone number and password, so that I can log in to the VeXeBus system and book bus tickets.		
Preconditions	The Customer role exists in the system. The backend server and database are available. The phone number has not already been registered.		
Postconditions	On Success: • A new user account is created. • The Customer role is assigned to the account. • Customer profile information is stored. • The user is redirected to the login page with a success message. On Failure: • No account information is stored. • The system displays an appropriate error message. • The user remains on the registration page.		
Normal Sequence/Flow	Normal Flow 1. The user navigates to /register. 2. The system displays the registration form. 3. The user enters a phone number and password. 4. The user confirms the password. 5. The user optionally enters CCCD and email information. 6. The user clicks the Create Account button. 7. The system validates the registration information. 8. The system checks whether the phone number already exists. 9. The system creates the user account. 10. The system assigns the Customer role to the account. 11. The system creates the customer profile. 12. The system displays a registration success message. 13. The system redirects the user to /login.		
Alternative Sequences/Flows	A1: Duplicate Phone Number (Step 8) 1. The system detects that the phone number already exists. 2. The system displays an error message. 3. The user remains on the registration page. A2: Invalid Registration Information (Step 7) 1. 3. The system detects invalid input such as: ○ Passwords do not match ○ Invalid phone number format ○ Invalid CCCD format ○ Invalid email format 2. The system displays validation errors.		

	3. The registration process stops until the user corrects the information.
Exceptional Flow	E1: Network or Server Failure 1. The registration request cannot be completed because of a network or server issue. 2. The system displays a registration failure message. 3. The user remains on the registration page. E2: System Error During Registration 1. An unexpected system or database error occurs during registration. 2. The system cancels the registration process. 3. The system displays an error message. 4. No incomplete account data is stored.

2.2.3 Logout System

Use case ID, Name	UC-03: User Logout		
Primary Actors	Ticket Staff, Customer	Secondary Actors	System, Database
Description	A logged-in user creates a new cargo shipping order by entering sender information, receiver information, and one or more items with their quantities and weights. The system calculates the total price based on current cargo type prices, saves the order to the database, and returns a unique order code.		
User Story	As a ticket staff member, I want to create a new shipping order so that I can register cargo transportation requests from customers and generate a tracking code for them		
Preconditions	1. The user is logged into the system. 2. The backend API is running. 3. The database is connected. 4. Cargo types and their current prices exist in the database.		
Postconditions	On Success: • The user session is cleared. • Authentication data is removed from browser storage. • The system returns to the login page. • The interface updates to show logged-out options. On Failure: • No special failure handling is required because clearing an empty session does not cause errors.		
Normal Sequence/Flow	Normal Flow 1. The user clicks the Logout button. 2. The system clears the current user session. 3. The system removes stored authentication information. 4. The system updates the application state to reflect that the user is logged out. 5. The system redirects the user to /login. 6. The interface re-renders and displays Login and Register options.		
Alternative Sequences/Flows	A1: Session Already Cleared 1. The system attempts to clear authentication data that does not exist.		

	2. No error occurs. 3. The logout process continues normally.
--	--

2.3 Hieu

2.3.1 Cargo types managing

Manager

Use case ID, Name	UC-04: Managing Cargo types		
Primary Actors	Manager	Secondary Actors	System, Database
Description	A manager can manage cargo types on manager's form, system read all types and prices added or updated		
User Story	As a manager, I want to: - Add a new cargo type including type and price per kg - Update cargo types and prices - Delete cargo types		
Preconditions	The Customer role exists in the system. The backend server and database are available. The phone number has not already been registered.		
Postconditions	On Success: - A new order is saved to the cargo_ticket table. - Order details are saved to the cargo_ticket_detail table. - A unique order code is generated (format: CG-YYYYMMDDHHMMSS-RANDOM). - The total price is calculated and stored. - The order status is set to "received". - The form is reset for the next order. . On Failure: - No data is saved to the database. - Error messages are displayed to the user. - The form retains the entered data for correction.		
Normal Sequence/Flow	Normal Flow 14. The user navigates to the "Create Order" page . 15. The user enters sender information (full name, phone number, ID number, email optional). 16. The user enters receiver information (full name, phone number, ID number, email optional). 17. The user selects a cargo type from the dropdown menu. 18. The user enters the specific item name, quantity, and weight per unit. 19. The user clicks "Add Item" to include more items (repeat steps 4-5 as needed). 20. The user can click "Remove" next to any item to delete it.		

	21. The user enters optional notes (e.g., "fragile", "handle with care"). 22. The user clicks the "Submit Order" button. 23. The system calls the API endpoint POST /api/cargo/order. The system creates the customer profile. 24. The system displays a success message with the generated order code. 25. The system resets all form fields.
Alternative Sequences/Flows	A1: Missing Required Fields 1. The user clicks "Submit Order" without filling all required fields (sender name, phone, ID, receiver name, phone, ID). 2. The system highlights the empty fields in red. 3. The system displays an error message: "Please fill in all required fields." 4. No API call is made. 5. The user corrects the missing information. 6. The normal flow resumes from step 9. A2: No Items Added 1. The user clicks "Submit Order" without adding any items to the order. 2. The system displays an error message: "Please add at least one valid item." 3. No API call is made. 4. The user adds items following the normal flow.
Exceptional Flow	E1: Network or Server Failure 4. The registration request cannot be completed because of a network or server issue. 5. The system displays a registration failure message. 6. The user remains on the registration page. E2: System Error During Registration 5. An unexpected system or database error occurs during registration. 6. The system cancels the registration process. 7. The system displays an error message. 8. No incomplete account data is stored.

2.3.2 Package type pricing

Ticket Staff

Use case ID, Name	UC-04: Create Cargo types and prices		
Primary Actors	Manager	Secondary Actors	System, Database
Description	Manager provides Cargo types and prices (CẦN ĐƯỢC SỬA LẠI!!!!)		
User Story	As a guest user, I want to register an account using my phone number and password, so that I can log in to the VeXeBus system and book bus tickets.		
Preconditions	The Customer role exists in the system. The backend server and database are available. The phone number has not already been registered.		
Postconditions	On Success: - A new order is saved to the cargo_ticket table. - Order details are saved to the cargo_ticket_detail table. - A unique order code is generated (format: CG-YYYYMMDDHHMMSS-RANDOM). - The total price is calculated and stored. - The order status is set to "received". - The form is reset for the next order. . On Failure: - No data is saved to the database. - Error messages are displayed to the user. - The form retains the entered data for correction.		
Normal Sequence/Flow	Normal Flow - The user navigates to the "Create Order" page . - The user enters sender information (full name, phone number, ID number, email optional). - The user enters receiver information (full name, phone number, ID number, email optional). - The user selects a cargo type from the dropdown menu. - The user enters the specific item name, quantity, and weight per unit. - The user clicks "Add Item" to include more items (repeat steps 4-5 as needed). - The user can click "Remove" next to any item to delete it. - The user enters optional notes (e.g., "fragile", "handle with care"). - The user clicks the "Submit Order" button. - The system calls the API endpoint POST /api/cargo/order. The system creates the customer profile. - The system displays a success message with the generated order code. - The system resets all form fields.		

Alternative Sequences/Flows	A1: Missing Required Fields - The user clicks "Submit Order" without filling all required fields (sender name, phone, ID, receiver name, phone, ID). - The system highlights the empty fields in red. - The system displays an error message: "Please fill in all required fields." - No API call is made. - The user corrects the missing information. - The normal flow resumes from step 9. A2: No Items Added - The user clicks "Submit Order" without adding any items to the order. - The system displays an error message: "Please add at least one valid item." - No API call is made. - The user adds items following the normal flow.
Exceptional Flow	E1: Network or Server Failure - The registration request cannot be completed because of a network or server issue. - The system displays a registration failure message. - The user remains on the registration page. E2: System Error During Registration - An unexpected system or database error occurs during registration. - The system cancels the registration process. - The system displays an error message. - No incomplete account data is stored.

2.4 Voucher CRUD

2.4.1 Create Voucher

Use case ID, Name	UC-04: Create New Voucher		
Primary Actors	Manager	Secondary Actors	System, Database
Description	The Manager provides configuration details for a new discount campaign (code, discount type, values, validity dates, and usage limits) through the admin panel. The system validates the input parameters against business criteria and database rules before inserting the voucher record into the system.		
User Story	As an Operations Manager, I want to create new discount vouchers so that I can run promotional campaigns, attract more passengers, and boost trip booking revenue.		
Preconditions	- The Manager must be authenticated with an active session belonging to a 'MANAGER' role. - The backend server must be running and accessible. - The database connection to VeXeDB must be active.		

Postconditions	On success: 1. A new row is inserted into the voucher table with usedCount initialized to 0. 2. The audit fields createdBy and createdAt are populated with the Manager's ID and the current timestamp. 3. The system returns a success notification and redirects the Manager to the Voucher Dashboard view. On failure: 1. No data is committed to the database. 2. A specific validation error message is displayed highlighting the faulty fields. 3. The Manager remains on the creation form with their input preserved.
Normal Sequence/Flow	Normal Flow 1. The Manager navigates to the Voucher Management sub-panel. 2. The system displays the "Create New Voucher" form. 3. The Manager inputs the voucher code, discount type, value bounds, usage threshold, and active dates. 4. The Manager clicks the "Save Voucher" button. 5. The system validates that all fields match structural bounds and business constraints. 6. The system verifies that the voucherCode does not already exist in the database. 7. The system calculates and runs the insertion transaction onto the voucher table. 8. The system displays a success toast notification. 9. The system redirects the Manager back to the Voucher List View.
Alternative Sequences/Flows	A1 - Percent Discount Validation (Step 5) 1. The Manager selects discountType as 'PERCENT'. 2. The Manager inputs a value greater than 100 in the discountValue field. 3. The system catches the business rule infraction (percentages cannot exceed 100%). 4. The system blocks the submission and displays an error message next to the value field.
Exceptional Flow	Exceptional Flow E1 – Duplicate Voucher Code Rule (Step 6) 1. The system detects that the input voucherCode already exists in the voucher table. 2. The system throws a unique constraint violation error. 3. The system alerts the Manager: "This voucher code is already in use. Please choose a unique name."

	E2 – Date Inversion Error (Step 5) 1. The Manager sets an endEffectiveDate that occurs chronologically before the startEffectiveDate. 2. The system flags the violation against constraint CK_Voucher_Dates. 3. The submission is aborted, and a date validation warning is displayed.
--	--

2.4.2 Read Voucher List and Metrics

Use case ID, Name	UC-04: Read Voucher List and Metrics		
Primary Actors	Manager	Secondary Actors	System, Database
Description	The Manager views a comprehensive list of all generated voucher campaigns to inspect code distributions, dynamic conversion progress counters (usedCount vs usageLimit), and basic lifetime diagnostics.		
User Story	As an Operations Manager, I want to view a list of all active and expired vouchers so that I can monitor campaign absorption rates and inspect system metrics.		
Preconditions	1. The Manager must be authenticated with an active session belonging to a 'MANAGER' role. 2. The database connection to VeXeDB must be active.		
Postconditions	<i>On success:</i> 1. The Database sends all matched voucher records back to the System. 2. The System constructs a data grid rendering real-time consumption metrics (usedCount / usageLimit). <i>On failure:</i> 1. The System logs the connection exception and presents a "Data fetch failed" notification to the Manager.		
Normal Sequence/Flow	1. The Manager clicks on the "Voucher Management" tab. 2. The System dispatches an asynchronous HTTP GET request to the backend API. 3. The System executes a selection query against the Database for rows in the voucher table. 4. The Database pulls the records and forwards the data stream back to the System. 5. The System calculates runtime status flags (e.g., "Active", "Expired", "Exhausted") by comparing current server time against endEffectiveDate and comparing usedCount with usageLimit. 6. The System binds the dataset to the user interface grid. 7. The System renders the populated dashboard for the Manager.		
Alternative Sequences/Flows			

Exceptional Flow	
------------------	--

2.4.3 Update Existing Voucher

Use case ID, Name	UC-04: Read Voucher List and Metrics		
Primary Actors	Manager	Secondary Actors	System, Database
Description	The Manager selects a targeted voucher campaign and alters configurable settings—such as increasing allocation pools (usageLimit) or extending date windows (endEffectiveDate).		
User Story	As an Operations Manager, I want to edit voucher properties so that I can dynamically adjust promotional limits or extend campaigns based on market performance.		
Preconditions	1. The target voucher record must already exist within the voucher table. 2. The Manager must be authenticated under administrative clearance.		
Postconditions	On success: 1. Changes are safely stored within the database row. 2. The audit properties updatedAt and updatedBy are automatically overwritten. 3. Updated parameters take effect instantly across the checking/validation endpoints. On failure: 1. The target database record remains unmodified. 2. An informative error message explains the failure criteria.		
Normal Sequence/Flow	1. The Manager locates a voucher row and clicks the "Edit" action button. 2. The system retrieves the single object blueprint from the database and fills out an editing form. 3. The Manager modifies parameters (e.g., raises the usageLimit or delays the endEffectiveDate). 4. The Manager clicks the "Apply Updates" button. 5. The system runs logical validation assessments to ensure metrics comply with table constraints. 6. The system executes an atomic UPDATE statement on the voucher table. 7. The system sends a success alert and returns the Manager to the list view.		
Alternative Sequences/Flows	A1 - Locked Immutable Fields (Step 3) 1. The system checks if the targeted voucher's ID is already mapped to confirmed transactions inside the passenger ticket table.		

	- If mapped, the system permanently locks and disables editing controls on the voucherCode and discountValue fields to prevent historical bookkeeping errors. - The Manager can still modify mutable parameters like usageLimit or endEffectiveDate.
Exceptional Flow	

Use case ID, Name	UC-04: Read Voucher List and Metrics		
Primary Actors	Manager	Secondary Actors	System, Database
Description	The Manager selects a targeted voucher campaign and alters configurable settings—such as increasing allocation pools (usageLimit) or extending date windows (endEffectiveDate).		
User Story	As an Operations Manager, I want to edit voucher properties so that I can dynamically adjust promotional limits or extend campaigns based on market performance.		
Preconditions	- The target voucher record must already exist within the voucher table. - The Manager must be authenticated under administrative clearance.		
Postconditions	On success: - Changes are safely stored within the database row. - The audit properties updatedAt and updatedBy are automatically overwritten. - Updated parameters take effect instantly across the checking/validation endpoints. On failure: - The target database record remains unmodified. - An informative error message explains the failure criteria.		
Normal Sequence/Flow	- The Manager locates a voucher row and clicks the "Edit" action button. - The system retrieves the single object blueprint from the database and fills out an editing form. - The Manager modifies parameters (e.g., raises the usageLimit or delays the endEffectiveDate). - The Manager clicks the "Apply Updates" button. - The system runs logical validation assessments to ensure metrics comply with table constraints. - The system executes an atomic UPDATE statement on the voucher table.		

	14. The system sends a success alert and returns the Manager to the list view.
Alternative Sequences/Flows	A1 - Locked Immutable Fields (Step 3) 4. The system checks if the targeted voucher's ID is already mapped to confirmed transactions inside the <code>passenger_ticket</code> table. 5. If mapped, the system permanently locks and disables editing controls on the <code>voucherCode</code> and <code>discountValue</code> fields to prevent historical bookkeeping errors. 6. The Manager can still modify mutable parameters like <code>usageLimit</code> or <code>endEffectiveDate</code>.
Exceptional Flow	<i>E1 – Concurrency Conflict / Stale Data Overwrite (Step 7):</i> 1. Two Managers open the same voucher detail card concurrently. Manager A extends the date and clicks save. 2. Manager B leaves their screen open, tries to change an allocation threshold moments later, and clicks save using stale historical entity states. 3. The System or Database intercepts the query using timestamp matching/version checks and discovers a concurrency mismatch. 4. The Database rejects the update; the System forces an entity state refresh and outputs: "This record has been modified by another administrator. Your inputs have been reset to match current data settings." <i>E2 – Historical Constraint Regression Violation (Step 7):</i> 1. The Manager attempts to scale back the <code>usageLimit</code> to a number that falls below the count of what passengers have already claimed (<code>usageLimit < usedCount</code>). 2. The Database tests the condition against constraint <code>CK_Voucher_Limits</code>. 3. The Database rejects the change as mathematically inconsistent and fails the update query. 4. The System intercepts the exception and explains to the Manager: "The total allocation ceiling cannot be adjusted below the number of vouchers already redeemed by customers."

2.4.4 Delete / Deactivate Voucher

Use case ID, Name	UC-04: Read Voucher List and Metrics		
Primary Actors	Manager	Secondary Actors	System, Database

Description	The Manager removes an unused voucher from the dashboard or permanently forces a retirement/deactivation on a live voucher code to prevent passengers from utilizing it during checkout workflows.
User Story	As an Operations Manager, I want to delete or deactivate a voucher code so that I can terminate an underperforming or compromised promotional campaign immediately.
Preconditions	1. The designated target record must exist inside the voucher table.
Postconditions	On success (Physical Deletion): 1. The targeted record row is entirely expunged from the database. 2. The row is removed from the administration interface grid. On success (Logical Deactivation): 1. The voucher row remains in the database to preserve historical linkages. 2. The endEffectiveDate is set to the current time stamp (GETDATE()), preventing any future checkout validation passes.
Normal Sequence/Flow	1. The Manager selects a voucher and clicks the "Delete" button. 2. The system prompts the Manager with a confirmation modal: "Are you sure you want to remove this item?". 3. The Manager clicks "Confirm". 4. The system checks database relational constraints to see if the voucherId has already been linked to records inside the passenger_ticket table. 5. The system confirms no dependencies exist. 6. The system sends a physical execution statement (DELETE FROM [voucher] WHERE voucherId = @id). 7. The system updates the UI and removes the row from the display grid.
Alternative Sequences/Flows	A1 - Soft Deactivation Safeguard (Step 5) 1. The system discovers that the voucherId is currently attached to existing ticket orders inside passenger_ticket. 2. The system catches that a hard physical deletion would violate foreign key constraints and break historical reporting. 3. The system transparently switches execution strategy to an automatic logical deactivation: it updates the voucher's endEffectiveDate to the current timestamp. 4. The system alerts the Manager: "Voucher has historical records and cannot be deleted. It has been automatically closed and deactivated instead."
Exceptional Flow	<i>E1 – Cascade Integrity Violation Block (Step 6):</i> 1. The System bypasses or fails to run the defensive lookup routine in Step 4 and directly issues a raw physical delete execution statement (DELETE

	FROM [voucher] WHERE voucherId = @id) against a voucher containing historical transactions. 2. The database engine catches the foreign key integrity violation immediately from passenger_ticket which references [voucher]([voucherId]). 3. The database engine halts the processing chain, blocks the execution completely, and raises a foreign key violation state exception. 4. The System safely traps the database error, halts application workflow regression, cancels UI state updates, and displays: "Critical error: The system blocked this deletion because it is actively tied to sales data. Action aborted."
--	--

2.4.5 Delete / Deactivate Voucher

Use case ID, Name	UC05 - Manage Trip Schedule		
Primary Actors	Manager	Secondary Actor	System database
Description	As a Manager, I want to have a system that can view all the available coach, view all available coach stop, view all time stamps so I can create, add, update, delete, view the trip that I want, view the list of the trip in the present and the future day so i can manage trip easier		
Preconditions	1. The user has accessed the Ralsei Coach House manager website 2. Active coach routes, schedules, seat inventories, stop points, are synchronized and available in the system. 3. Coach have "AVAILABLE" status		
Postconditions	On success (Physical Deletion): 1. The targeted record row is entirely expunged from the database. 2. the row is added, updated, displayed, removed from the staff interface grid. On success (Logical Deactivation): 3. The Trip row remains in the database to preserve historical linkages.		
Normal Sequences/Flow	1. The manager views the default available trip in the day first.		

	2. The Manager will select the route with the following coach, route stop, seat layout, seat type price to create, update, delete coach. 3. The manager will be ask to confirm the action 4. If the manager confirm The coach will be create, update, delete success 5. Use case end successfully
Alternative Sequences/Flows	Alternative Flow A1: if the manager not confirmed 1. The Coach create/update/delete will be cancel 2. Return to step 1.
Exceptional Flow	1. The database engine halts the processing chain, blocks the execution completely, and raises a foreign key violation state exception. 2. The System safely traps the database error, halts application workflow regression, cancels UI state updates, and displays: "Critical error: The system blocked this deletion because it is actively tied to sales data. Action aborted."

2.5 Cargo Tracking

2.5.1 Track Cargo

Use case ID, Name	UC-C06: Track Cargo		
Primary Actors	Customer	Secondary Actors	CargoTrackingService, Database
Description	The customer enters a tracking code to view detailed information about a shipped order, including its current status, sender/receiver information, pickup/dropoff points, trip details, and the list of goods with shipping fees.		
User Story			
Preconditions	1. The backend system is operational. 2. Order data already exists in the database (<code>cargo_ticket</code>, <code>cargo_ticket_detail</code> tables). 3. The API <code>/api/v1/cargo-tracking/{ticketCode}</code> allows public access (no authentication required).		
Postconditions	1. Full order information is displayed if found. 2. An error message is displayed if the code is not found or a system error occurs.		
Normal Sequence/Flow	1. The customer accesses the <code>/tra-cuu</code> page. 2. The system displays a tracking code input form.		

	3. The customer enters the tracking code (e.g., <code>CG_CODE_0001</code>). 4. The customer clicks the "Track" button. 5. The system calls <code>GET /api/v1/cargo-tracking/{ticketCode}</code>. 6. The backend looks up the <code>CargoTicket</code> by <code>ticketCode</code>. 7. The backend retrieves <code>CargoTicketDetail</code> by <code>cargoTicketId</code>. 8. The backend retrieves the pickup/dropoff point names from <code>CoachStop</code>. 9. The backend retrieves the trip information (<code>Trip + Route</code>). 10. The backend retrieves the unit of measure from <code>CargoTypePrice</code>. 11. The backend returns the <code>CargoTrackingResponse</code>. 12. The frontend renders the status timeline + information cards + goods table.
Alternative Sequences/Flows	A1: Order not yet assigned to a trip — At step 9, if <code>tripId = 0</code> or the Trip is not found, the backend returns <code>tripRouteName = null</code> and <code>tripDepartureTime = null</code>. The frontend displays "N/A". A2: Pickup/dropoff point does not exist — At step 8, if <code>pickupStopId</code> or <code>dropoffStopId</code> is not found in <code>CoachStop</code>, the backend returns the point name as "N/A". A3: COD = 0 — At step 12, if <code>codAmount = 0</code>, the frontend hides the COD line.
Exception Flow	E1: Tracking code does not exist — At step 6, <code>findByTicketCode</code> returns <code>Optional.empty()</code>. The backend throws a <code>ResourceNotFoundException</code>. The frontend receives a 404 status and displays "No order found with this code." E2: Backend connection error — At step 5, the request fails due to a network issue or server downtime. The frontend catches the error and displays "An error occurred, please try again later." E3: System error (500) — The server encounters an internal error. The frontend receives a 500 status and displays a generic error message.

2.5.2 View Order Detail After Tracking

Use case ID, Name	UC05 - Manage Trip Schedule		
Primary Actors	Customer	Secondary Actor	System database
User Story			
Description	After a successful tracking lookup, the customer views the order's detailed information, including the status timeline, sender/receiver information, pickup/dropoff points, trip details, and the list of goods with prices.		
Preconditions	1. The tracking code lookup was successful. 2. Order data exists.		
Postconditions	The customer understands the order's current status and detailed information.		

Normal Sequences/Flow	1. The system displays a 4-step status timeline: Received → Loaded onto vehicle → Arrived → Delivered. 2. The system displays 4 information cards: sender, receiver, pickup/dropoff point, trip. 3. The system displays an order summary card: tracking code, status, description, fee payer, COD, total fee. 4. The system displays a goods list table: description, quantity, weight (kg), volume (m³), unit of measure, amount.
Exceptional Flow	E1: No goods — If <code>items</code> = [], the goods table is hidden. E2: No trip assigned — If <code>tripRouteName</code> = <code>null</code>, display "N/A".

2.6 Manages Account

Use case ID, Name	UC05 - Manage Trip Schedule		
Primary Actors	Customer	Secondary Actor	System database
User Story			
Description	After a successful tracking lookup, the customer views the order's detailed information, including the status timeline, sender/receiver information, pickup/dropoff points, trip details, and the list of goods with prices.		
Preconditions	1. The tracking code lookup was successful. 2. Order data exists.		
Postconditions	The customer understands the order's current status and detailed information.		
Normal Sequences/Flow	1. The system displays a 4-step status timeline: Received → Loaded onto vehicle → Arrived → Delivered. 2. The system displays 4 information cards: sender, receiver, pickup/dropoff point, trip. 3. The system displays an order summary card: tracking code, status, description, fee payer, COD, total fee.		

	4. The system displays a goods list table: description, quantity, weight (kg), volume (m³), unit of measure, amount.
Exceptional Flow	E1: No goods — If items = [], the goods table is hidden. E2: No trip assigned — If tripRouteName = null, display "N/A".

3. Functional Requirements

[Provide descriptions about the system's functions/screens. The functions/screens are grouped by the system features, and even sub-features if needed. For the screens, you need to provide the screen layouts (mock-up screens) and relevant specifications if needed]

3.1 Feature Name1

3.1.1 SubFeature Name1.1

3.1.1.1 Screen/Function Name1

[Content #1: UI layout (Mockup screen prototype)]

[Content #2: brief descriptions of the screen/function, mapped to the relevant use cases]

[Content #3: provide further descriptions for the screen's components/fields using table format below]

Field Name	Description
Field Name1	Field description: data type min/max length or value, initial data, etc.
Field Name2	...
Field Group-Name1	
Field Name3	...
Field Name4	...
Field Group-Name2	
...	...

3.1.1.2 Screen/Function Name2

...

3.1.2 SubFeature Name1.2

...

3.2 User Authentication

3.2.1 User Register

...

3.2.2 User Login

...

3.2.3 Password Reset

...

3.3 System Administration

3.3.1 Master Data

3.3.1.1 Setting List

Setting List

(1)

(2)

All Types ▾

All Statuses ▾

Enter keyword(s) to search

Search

[New Setting](#)

Id	Name	Type ^	Value	Priority	Status	Action
1	Manager	User Role		2	Active	Edit Deactivate
3	Admin	User Role		1	Inactive	Edit Activate
4	Teacher	User Role		4	Active	Edit Deactivate (3)

<

1

2

...

9

10

>

This screen allows the Administrator to:

- View Setting List: view list of current master data.
- Filter Setting List: filter master data by data types, statuses
- Search Settings: enter keyword(s) to search master data by their names or values
- Sort Setting List: sort master data list (ascending, descending) by clicking column headers

On the screen, s/he can also

- Activate/Deactivate Setting: change status of a specific inactive/active master data
- Choose to go to the Setting Details screens for adding new or updating an existing master data by clicking the New Setting or Edit link.

Field Description

Field Name	Description
(1)	Initial values: all the active setting names with null or blank type Hover the mouse to show the field name: "Setting Type"
(2)	Initial values: All Statuses, Active, Inactive (default value "All Status") Hover the mouse to show the field name: "Setting Status"
(3)	The change-status action is Activate or Deactivate depending on the current status of the relevant setting (Inactive or Active, respectively).

3.3.1.2 Setting Details

Setting Details

Name*

Manager

Type

User Role

Value

Priority

2

Status

☐ Active ☒ Inactive

Description

Mô tả cho role giảng viên

Submit

This screen allows the Administrator to:

- Add New Setting: add new master data.
- Update Setting Details: update details of a specific master data

Field Description

Field Name	Description
Name	Data type: non-digit string, max length of 20 characters
Type	Initial data values: all active setting names (with null or blank type)
Value	Data type: any string, max length of 100 characters
Priority	Data type: a positive integer
Description	Data type: any string, max length of 200 characters

3.3.2 User Management

3.3.2.1 User List

...

3.3.2.2 User Details

...

4. Non-Functional Requirements

4.1 External Interfaces

4.1.1 User Interfaces

4.1.2 Hardware Interfaces

4.1.3 Software Interfaces

4.1.4 Communications Interfaces

Standard communication: HTTPS

4.2 Other Requirement

4.2.1 Usability requirement

4.2.2 Performance requirement

backend:

Database efficient: solve N+1 Query Problem

Maximum Database Connection Pooling 20

Encrypt crucial information(email, phone number) in cache

Frontend:

Optimized Seat Map Rendering

Debouncing API Calls:

Graceful Degradation: loading screen when database takes time to load

4.2.3 Security requirement

4.2.4 Software Quality Attributes

4.2.5 Business rules

5. Requirement Appendix

[Provide business rules, common requirements, or other extra requirements information here]

5.1 Business Rules

[Provide common business rules that you must follow. The information can be provided in the table format as the sample below]

ID	Rule Definition
BR-01	Delivery time windows are 15 minutes, beginning on each quarter hour.
BR-02	Deliveries must be completed between 10:00 A.M. and 2:00 P.M. local time, inclusive.
BR-03	All meals in a single order must be delivered to the same location.
BR-04	All meals in a single order must be paid for by using the same payment method.
BR-11	If an order is to be delivered, the patron must pay by payroll deduction.
BR-12	Order price is calculated as the sum of each food item price times the quantity of that food item ordered, plus applicable sales tax, plus a delivery charge if a meal is delivered outside the free delivery zone.
BR-24	Only cafeteria employees who are designated as Menu Managers by the Cafeteria Manager can create, modify, or delete cafeteria menus.
BR-33	Network transmissions that involve financial information or personally identifiable information require 256-bit encryption.
BR-86	Only regular employees can register for payroll deduction for any company purchase.
BR-88	An employee can register for payroll deduction payment of cafeteria meals if no more than 40 percent of his gross pay is currently being deducted for other reasons.

5.2 System Messages

#	Message code	Message Type	Context	Content
1	MSG01	In line	There is not any search result	<i>No search results.</i>
2	MSG02	In red, under the text box	Input-required fields are empty	<i>The * field is required.</i>
3	MSG03	Toast message	Updating asset(s) information successfully	<i>Update asset(s) successfully.</i>

4	MSG04	Toast message	Adding new asset successfully	<i>Add asset successfully.</i>
5	MSG05	Toast message	Confirming email of asset hand-over is sent successfully	<i>A confirmation email has been sent to {email_address}.</i>
6	MSG06	Toast message	Resetting asset information successfully	<i>Return asset(s) successfully.</i>
7	MSG07	Toast message	Deleting asset information successfully	<i>Delete asset(s) successfully.</i>
8	MSG08	In red, under the text box	Input value length > max length	<i>Exceed max length of {max_length}.</i>
9	MSG09	In line	Username or password is not correct when clicking sign-in	<i>Incorrect user name or password. Please check again.</i>
10	..			

5.3 Other Requirements...